WYDZIAŁ ELEKTRONIKI I TECHNIK INFORMACYJNYCH

Sieci Wielousługowe (SWUS)

Projekt

Jakub Surma 327768

Albert Kołsut 331493

27 marca 2025, Warszawa

Spis treści

1. Utworzenie klastrów Kubernetesa (K8S) 2
1.1. Wykorzystywane technologie 2
1.2. Konfiguracja środowiska i instalacja klastra Kubernetes 2
1.3. Weryfikacja działania 3
2. Orkiestracja funkcji sieciowych 3
2.1. Orkiestracja sieci 5G z wykorzystaniem Helm 3
2.2. Weryfikacja działania 4
3. Uruchomienie sieci 4

1. Utworzenie klastrów Kubernetesa (K8S)

1.1. Wykorzystywane technologie

Do utworzenia klastra Kubernetes wykorzystaliśmy zestaw technologii, których wersje zostały dobrane w sposób zapewniający wzajemną kompatybilność oraz stabilność działania całego środowiska. Szczególną uwagę zwróciliśmy na możliwość integracji komponentów sieciowych niezbędnych do uruchomienia i testowania architektury sieci 5G, wykorzystywanych w drugim etapie projektu.

Komponent	**Wersja**
VirtualBox	7.1
Ubuntu Server LTS	20.04.6
Docker	20.10.24
Kubernetes	1.26.15
cri-dockerd	0.3.21.3
Calico	3.26.4
Multus	4.0.2

Tabela 1. Zestaw wykorzystanych technologii

1.2. Konfiguracja środowiska i instalacja klastra Kubernetes

1. **Wyłączenie mechanizmu swap**
   Mechanizm pamięci wymiany (swap) został wyłączony na wszystkich węzłach klastra, ponieważ Kubernetes wymaga pełnej kontroli nad zarządzaniem pamięcią operacyjną. Aktywna pamięć swap może prowadzić do nieprzewidywalnego zachowania kubeleta oraz problemów z planowaniem zasobów.

2. **Instalacja Dockera**
   Docker został zainstalowany jako silnik kontenerowy odpowiedzialny za uruchamianie i zarządzanie kontenerami. Zapewnia on środowisko wykonawcze dla aplikacji kontenerowych, które są podstawową jednostką wdrożeniową w Kubernetes.

3. **Instalacja cri-dockerd**
   Komponent cri-dockerd umożliwia integrację Dockera z Kubernetes poprzez interfejs Container Runtime Interface (CRI). Jest on wymagany, ponieważ Docker nie jest już natywnie wspieranym środowiskiem uruchomieniowym w Kubernetes.

4. **Instalacja narzędzi Kubernetes**
   Zainstalowano narzędzia kubelet, kubeadm oraz kubectl. Kubelet odpowiada za zarządzanie kontenerami na poziomie węzła, kubeadm służy do inicjalizacji i konfiguracji klastra, natomiast kubectl umożliwia administrację i monitorowanie zasobów klastra.

5. **Inicjalizacja klastra Kubernetes (master)**
   Na węźle głównym przeprowadzono inicjalizację klastra przy użyciu narzędzia kubeadm. Podczas tego procesu zdefiniowano zakresy adresów IP dla sieci podów oraz usług, a także wskazano gniazdo CRI dla cri-dockerd.

6. **Dołączenie węzłów roboczych (worker)**
   Węzły robocze zostały dołączone do klastra za pomocą unikalnej komendy kubeadm join wygenerowanej na węźle master. Każdy węzeł roboczy pełni rolę środowiska uruchomieniowego dla aplikacji kontenerowych.

7. **Konfiguracja dostępu administracyjnego**
   Skonfigurowano plik konfiguracyjny kubectl, co umożliwiło zarządzanie klastrem z poziomu użytkownika administracyjnego oraz weryfikację poprawności działania klastra.

8. **Instalacja wtyczki sieciowej Calico**
   Calico zostało wdrożone jako wtyczka CNI odpowiedzialna za komunikację sieciową pomiędzy podami w klastrze, routing pakietów oraz egzekwowanie polityk sieciowych.

9. **Instalacja Multus CNI**
   Multus umożliwia przypisanie wielu interfejsów sieciowych do pojedynczego poda, co pozwala na realizację bardziej zaawansowanych scenariuszy sieciowych.

1.3. Weryfikacja działania

Korzystając z komendy **kubectl get nodes** sprawdzamy czy klaster jest sprawny. Jak widać wszystkie maszyny są w stanie **Ready** co oznacza, że klaster działa poprawnie.

```
master@master:~$ kubectl get nodes
NAME      STATUS   ROLES           AGE     VERSION
master    Ready    control-plane   5d22h   v1.26.15
worker1   Ready    <none>          5d1h    v1.26.15
worker2   Ready    <none>          5d1h    v1.26.15
```

Rysunek 1. kubectl get nodes

2. Orkiestracja funkcji sieciowych

2.1. Orkiestracja sieci 5G z wykorzystaniem Helm

1. **Dodanie repozytorium Helm projektu Towards5GS**
   Do środowiska Helm dodano repozytorium projektu Towards5GS, które dostarcza gotowe Helm Charty umożliwiające wdrożenie elementów sieci 5G, w tym rdzenia sieci Free5GC oraz symulatora UERANSIM. Repozytorium to zapewnia spójne i przetestowane konfiguracje dostosowane do środowiska Kubernetes.

2. **Weryfikacja dostępności chartów Helm**
   Poprawność dodania repozytorium została potwierdzona poprzez sprawdzenie listy dostępnych chartów, co umożliwiło weryfikację dostępności komponentów Free5GC oraz UERANSIM.

3. **Utworzenie dedykowanej przestrzeni nazw (namespace)**
   W celu logicznego oddzielenia komponentów sieci 5G od pozostałych zasobów klastra utworzono osobny namespace. Takie podejście ułatwia zarządzanie zasobami oraz monitorowanie stanu wdrożonych aplikacji.

4. **Instalacja rdzenia sieci 5G (Free5GC)**
   Rdzeń sieci 5G został wdrożony przy użyciu Helm Chart Free5GC. Podczas instalacji automatycznie uruchomiono wszystkie niezbędne funkcje sieciowe (AMF, SMF, UPF, NRF, AUSF, UDM, UDR, PCF) oraz bazę danych MongoDB, a także utworzono wymagane zasoby Kubernetes, takie jak Deployments i ConfigMapy.

5. **Weryfikacja poprawności uruchomienia rdzenia 5G**
   Po zakończeniu instalacji monitorowano stan podów w dedykowanej przestrzeni nazw. Rdzeń sieci uznano za poprawnie uruchomiony w momencie, gdy wszystkie pody Free5GC oraz MongoDB osiągnęły stan *Running*.

6. **Instalacja symulatora UERANSIM (gNB i UE)**
   Po uruchomieniu rdzenia sieci 5G wdrożono symulator UERANSIM, który pełni rolę stacji bazowej gNB oraz użytkownika końcowego UE. Umożliwia on testowanie działania sieci 5G bez konieczności użycia fizycznej infrastruktury radiowej.

7. **Integracja UERANSIM z Free5GC**
   Podczas instalacji UERANSIM skonfigurowano połączenie z funkcją AMF w rdzeniu sieci Free5GC, co umożliwiło rejestrację symulowanego użytkownika UE oraz przeprowadzenie podstawowych procedur sygnalizacyjnych charakterystycznych dla sieci 5G.

2.2. Weryfikacja działania

Korzystając z komendy **kubectl get pods** sprawdzamy czy klaster jest sprawny. Jak widać wszystkie procesy są w stanie **Running** co oznacza, że funkcje sieciowe zostały skonfigurowane poprawnie.

```
master@master:~$ kubectl get pods -A
NAMESPACE            NAME                                       READY   STATUS    RESTARTS        AGE
5g                   free5gc-free5gc-amf-amf-749cb75cb5-psfbk   1/1     Running   4 (8m37s ago)   3d
5g                   free5gc-free5gc-ausf-ausf-7c949bb896-kgmc9 1/1     Running   4 (9m51s ago)   3d
5g                   free5gc-free5gc-dbpython-dbpython-c6b7c8558-69rrn 1/1 Running 4 (9m51s ago) 3d
5g                   free5gc-free5gc-nrf-nrf-5d7db6d8d5-4rkst   1/1     Running   4 (9m51s ago)   3d
5g                   free5gc-free5gc-nssf-nssf-565bc8d548-7shfw 1/1     Running   0               83s
5g                   free5gc-free5gc-pcf-pcf-fb7497ff9-wgzpq    1/1     Running   4 (9m51s ago)   3d
5g                   free5gc-free5gc-smf-smf-5d6987b66b-ld6wd   1/1     Running   2 (9m51s ago)   19h
5g                   free5gc-free5gc-udm-udm-7d96b9f499-4v2ps   1/1     Running   4 (9m51s ago)   3d
5g                   free5gc-free5gc-udr-udr-c87987489-sbnjj    1/1     Running   0               82s
5g                   free5gc-free5gc-upf-upf-6dcdcdd689-bzmqz   1/1     Running   1 (8m37s ago)   55m
5g                   free5gc-free5gc-webui-webui-56dc574fb9-75w5p 1/1   Running   4 (8m37s ago)   3d
5g                   mongodb-0                                  1/1     Running   4 (8m37s ago)   3d
5g                   ueransim-gnb-6bddb576dc-jv2gs              1/1     Running   2 (9m51s ago)   20h
5g                   ueransim-ue-7d586fcff8-qq2jm               1/1     Running   2 (9m30s ago)   121m
kube-system          calico-kube-controllers-6d6dc86d84-pskng   1/1     Running   4 (9m51s ago)   3d1h
kube-system          calico-node-4w8j4                          1/1     Running   4 (9m51s ago)   3d1h
kube-system          calico-node-9157b                          1/1     Running   4 (10m ago)     3d1h
kube-system          calico-node-v8wz9                          1/1     Running   4 (8m37s ago)   3d1h
kube-system          coredns-787d4945fb-gs627                   1/1     Running   4 (9m51s ago)   3d22h
kube-system          coredns-787d4945fb-xjprg                   1/1     Running   4 (9m51s ago)   3d22h
kube-system          etcd-master                                1/1     Running   5 (10m ago)     3d22h
kube-system          kube-apiserver-master                      1/1     Running   5 (10m ago)     3d22h
kube-system          kube-controller-manager-master             1/1     Running   5 (10m ago)     3d22h
kube-system          kube-multus-ds-85zjd                       1/1     Running   1 (8m37s ago)   46m
kube-system          kube-multus-ds-brjt6                       1/1     Running   4 (10m ago)     3d1h
kube-system          kube-multus-ds-l8chj                       1/1     Running   4 (9m51s ago)   3d1h
kube-system          kube-proxy-hn59t                           1/1     Running   5 (10m ago)     3d22h
kube-system          kube-proxy-s7czl                           1/1     Running   4 (8m37s ago)   3d1h
kube-system          kube-proxy-tb2gm                           1/1     Running   4 (9m51s ago)   3d2h
kube-system          kube-scheduler-master                      1/1     Running   5 (10m ago)     3d22h
local-path-storage   local-path-provisioner-c6977879-cd9xd      1/1     Running   4 (9m51s ago)   3d1h
```

Rysunek 2. kubectl get pods -A

3. Uruchomienie sieci

By sprawdzić czy wszystko działa poprawnie skorzystaliśmy z komendy **ping** oraz wykorzystaliśmy UERANSIM. Celem pingu była witryna google.com.

```
master@master:~$ kubectl -n 5g exec -it pod/ueransim-ue-7d586fcff8-qq2jm -- ping -c 4 google.com
PING google.com (142.251.98.139) 56(84) bytes of data.
64 bytes from nt-in-f139.1e100.net (142.251.98.139): icmp_seq=1 ttl=110 time=12.6 ms
64 bytes from nt-in-f139.1e100.net (142.251.98.139): icmp_seq=2 ttl=110 time=8.95 ms
64 bytes from nt-in-f139.1e100.net (142.251.98.139): icmp_seq=3 ttl=110 time=20.2 ms
64 bytes from nt-in-f139.1e100.net (142.251.98.139): icmp_seq=4 ttl=110 time=8.20 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
rtt min/avg/max/mdev = 8.200/12.501/20.227/4.765 ms
```

Rysunek 3. ping